Table des matières

Cours 10 — Passer à un format de table ouvert 1
Ce que vous allez faire 1
Avant de commencer 2
Étape 1 — Ce qu'un dossier de Parquet ne sait pas faire · RAW 2
Étape 2 — Écrire `iceberg.py` · RAW 3
Étape 3 — Ranger les exports · RAW 4
Étape 4 — Le catalogue et ce que la table a écrit · RAW 6
Étape 5 — Remonter au 12 juin · RAW 7
Étape 6 — Lire la table avec DuckDB, contre le bronze · RAW 8
Étape 7 — Versionner · RAW 8
À retenir 8

Cours 10 — Passer à un format de table ouvert

Cours 10/12 · La Brûlerie — Formation Data Engineering

Sources	**Ingestion**	**RAW**
d'où vient la donnée	aller la chercher, et la charger	la déposer, la ranger, la traiter
		✓ ici

Ce que vous allez faire

Le bronze du cours 02 est un dossier de Parquet : il dit ce qu'il contient maintenant, et rien de plus. Il ne garde que le dernier export de chaque jour, sans trace du premier export du 12 juin, et un lecteur qui le lit pendant qu'on le refait voit un dossier à moitié écrit. Un format de table ouvert ajoute aux fichiers Parquet des métadonnées qui disent lesquels comptent : chaque écriture fait un instantané, validé d'un coup par un catalogue, et chaque instantané se relit plus tard. Vous écrivez `iceberg.py`, qui range les exports de la caisse un à un dans une table Iceberg, puis vous remontez au 12 juin d'avant son second export, et vous lisez la table avec DuckDB, contre le bronze.

© 2026 iAlexMG. Tous droits réservés.

les exports de la caisse → `iceberg.py`, un instantané par export → le catalogue et l'historique de la table → le 12 juin, avant et après son second export → la table lue par DuckDB, contre le bronze du cours 02

Le code complet est dans `Codes_10_DE.md`, à ouvrir après avoir essayé.

Avant de commencer

Deux fenêtres de l'Anaconda Prompt servent à ce cours, toutes deux placées dans `Projet_Brulerie` : la première pour le Bloc-notes, Python et Git ; la seconde pour le notebook, qui s'ouvre à l'étape 4. Dans la première :

```
conda activate formations
REM Attendu : (formations) au début de l'invite.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
python -c "import pyiceberg; print(pyiceberg.__version__)"
REM Attendu : 0.12.0, ou plus récent de la même série.
```

PyIceberg, la bibliothèque Python qui écrit les tables Iceberg, est dans l'environnement `formations` depuis l'installation, par le `requirements.txt` de la formation. S'il manque : `pip install -r ..\01_Data_Engineering\requirements.txt`. Ce cours part aussi du bronze de la caisse, `plateforme\lac\bronze\tickets`, que le pipeline refait depuis le cours 05. S'il manque, reprenez la fin du cours 08.

Étape 1 — Ce qu'un dossier de Parquet ne sait pas faire · RAW

1.1 Lister le dossier du 12 juin dans le bronze. Le chemin va entre guillemets : sans eux, l'Anaconda Prompt coupe au signe `=`, et répond `Fichier introuvable`.

```
dir /b "plateforme\lac\bronze\tickets\jour=2025-06-12"
REM Attendu : data_0.parquet, un seul fichier.
```

Un seul fichier, celui du second export, et c'était voulu : le dernier export d'un jour fait foi. Mais plus personne ne peut dire ce que le bronze contenait le 12 au soir, avant que l'export repris arrive. Une table ouverte règle ce qu'un dossier ne sait pas faire :

La question	Le dossier de Parquet	La table Iceberg
ce que la table contient	les fichiers présents dans le dossier	les fichiers que ses métadonnées courantes nomment
une écriture	des fichiers effacés, puis écrits, l'un après l'autre	des fichiers ajoutés, puis un seul chemin changé dans le catalogue
un lecteur pendant l'écriture	ce qui est là, fichier à moitié écrit compris	l'état d'avant, entier, jusqu'au changement du chemin

© 2026 iAlexMG. Tous droits réservés.

La question	Le dossier de Parquet	La table Iceberg
l'état d'hier	perdu	un instantané, qui se relit

Iceberg est le format de ce cours ; Delta Lake, son voisin, fait la même chose par un journal de transactions, et reste à Boréal.

Étape 2 — Écrire `iceberg.py` · RAW

Le catalogue est une base qui tient, pour chaque table, le chemin de ses métadonnées courantes. Ici une base SQLite, `catalogue.db`, rangée dans le lac avec les fichiers de la table, dans `plateforme\lac\iceberg\` ; en entreprise, le catalogue est un service à part, et les fichiers vivent dans le lac. La table s'y range dans un espace de noms, `bronze`, comme une table dans le schéma d'une base. Quatre décisions font le script :

- les types sont ceux du bronze, les montants en `DECIMAL` : `COLONNES` reprend ceux de `sql/bronze.sql`, en dictionnaire Python ;
- la table naît au premier export, partitionnée par jour, comme le bronze : un dossier par jour ;
- chaque export écrase son jour. `dynamic_partition_overwrite` remplace les partitions que les données portent, et laisse les autres : le dernier export d'un jour fait foi, comme au bronze, mais l'ancien reste lisible ;
- chaque écriture porte dans son résumé l'export qu'elle range, ce qui permet de sauter un export que l'historique connaît déjà : relancé, le script n'écrit rien.

Pour remplir	L'outil
le catalogue	`SqlCatalog`, avec l'adresse de la base, `sqlite:///plateforme/lac/iceberg/catalogue.db`, et le dossier des fichiers, `warehouse`
lire un export, typé	`duckdb.read_csv(chemin, dtype=COLONNES, filename=True)`, puis `.select(…)` pour le jour, et `.to_arrow_table()`, qui le rend en Arrow, le format en mémoire que DuckDB et PyIceberg partagent
la table existe-t-elle	`catalogue.table_exists("bronze.tickets")`
la partitionner par jour	`table.update_spec().add_identity("jour").commit()` : la partition est la valeur même de `jour`
l'export d'un instantané	`s.summary.get("export")`, pour chaque `s` de `table.snapshots()`
écraser le jour d'un export	`table.dynamic_partition_overwrite(tickets, snapshot_properties={…})`

© 2026 iAlexMG. Tous droits réservés.

2.1 Créer le fichier, dans la première fenêtre.

```
notepad 01_de\iceberg.py
```

Accepter la création, puis écrire le code à partir de ce pseudocode, les trous ____ compris :

```
# 01_de/iceberg.py : les tickets de la caisse dans une table Iceberg, un instantané par export
from pathlib import Path
import duckdb
from pyiceberg.catalog.sql import SqlCatalog

COLONNES = {"ticket_id": "INTEGER", "horodatage": "TIMESTAMP", "type_ticket": "VARCHAR", "statut": "VARCHAR",
            "ticket_origine": "INTEGER", "membre_id": "VARCHAR", "sous_total": "DECIMAL(10,2)",
            "tps": "DECIMAL(10,2)", "tvq": "DECIMAL(10,2)", "total": "DECIMAL(10,2)",
            "mode_paiement": "VARCHAR", "arrondi": "DECIMAL(10,2)", "encaisse": "DECIMAL(10,2)"}
# 1. le catalogue SQL, une base SQLite, et les fichiers de la table : l'un et l'autre dans le lac
catalogue = SqlCatalog("brulerie", uri="sqlite:///____",
                       warehouse=Path("plateforme/lac/iceberg").resolve().as_posix())
catalogue.create_namespace_if_not_exists("bronze")

for fichier in sorted(Path("sources/caisse").glob("*/tickets_*.csv")):
    # 2. l'export, typé comme au bronze, avec son chemin et son jour livré
    jour = fichier.stem.split("_")[1]
    # 3. la table naît au premier export, partitionnée par jour, comme le bronze
    if not catalogue.table_exists("bronze.tickets"):
        table = catalogue.create_table("bronze.tickets", schema=tickets.schema)
        table.update_spec().add_identity("____").commit()
    table = catalogue.load_table("bronze.tickets")
    # 4. sauter un export que l'historique de la table connaît déjà
    if fichier.name in [s.summary.get("____") for s in table.snapshots()]:
        continue
    # 5. l'export écrase son jour : le dernier export d'un jour fait foi
    table.____(tickets, snapshot_properties={"export": fichier.name})
    print(f"{fichier.name} : {len(tickets)} tickets, {len(table.snapshots())} instantanés")
```

Enregistrer, puis fermer le Bloc-notes. Le chemin du lac passe par `resolve().as_posix()`, un chemin complet à barres obliques : c'est la forme que PyIceberg accepte sous Windows.

Étape 3 — Ranger les exports · RAW

3.1 Créer le dossier de la table, puis lancer le script, dans la première fenêtre. SQLite crée la base du catalogue, mais pas le dossier qui la porte.

© 2026 iAlexMG. Tous droits réservés.

```
mkdir plateforme\lac\iceberg
python 01_de\iceberg.py
REM Attendu : un avertissement, UserWarning: Delete operation did not match any records, puis
REM 31 lignes, une par export, de tickets_2025-06-01_202506011707.csv : 49 tickets, 1 instantanés
REM à tickets_2025-06-30_202506301807.csv : 33 tickets, 32 instantanés.
```

Écraser un jour, c'est d'abord effacer ce que la table en avait, puis ajouter l'export. Au 1er juin, il n'y avait rien à effacer, et PyIceberg le signale ; Python ne répète pas un même avertissement, si bien que les jours neufs suivants se taisent. Lire les lignes du 12 juin :

Export	Tickets	Instantanés
tickets_2025-06-11_202506111807.csv	48	11
tickets_2025-06-12_202506121542.csv	58	12
tickets_2025-06-12_202506130705.csv	65	14
tickets_2025-06-13_202506131807.csv	54	15

Le second export du 12 juin a fait deux instantanés, et non un : le jour effacé, puis le jour neuf. 31 exports, 32 instantanés.

3.2 Relancer le script.

```
python 01_de\iceberg.py
REM Attendu : rien ne s'affiche : l'historique de la table connaît les 31 exports.
```

Si la commande s'arrête sur	La cause, et le geste
`sqlalchemy.exc.OperationalError: (sqlite3.OperationalError) unable to open database file`	le dossier `plateforme\lac\iceberg` manque : faire le `mkdir`, puis la commande
`ModuleNotFoundError: No module named 'pyiceberg'`	PyIceberg manque à l'environnement : `pip install -r ..\01_Data_Engineering\requirements.txt`, puis la commande
`NameError: name 'tickets' is not defined`	la lecture de l'export, à l'étape 2 du script, manque ou porte un autre nom
`Binder Error: COLUMN_TYPES error: Columns with names: "…" do not exist in the CSV File`	un nom de `COLONNES` mal écrit : le dictionnaire des sources, section 1, donne les colonnes de `tickets`, et `sql/bronze.sql` les mêmes types
`FileNotFoundError: [WinError 3] Failed to open local file '…'`	le chemin complet d'un fichier de la table dépasse 260 caractères, la limite de Windows : rapprocher `Data_Domain` de la racine du disque

© 2026 iAlexMG. Tous droits réservés.

Étape 4 — Le catalogue et ce que la table a écrit · RAW

DuckDB lit une table Iceberg par le chemin de ses métadonnées, mais ne parle pas à un catalogue SQL. Le catalogue n'est pourtant qu'une base SQLite : DuckDB l'attache comme une autre, et y lit ce chemin.

4.1 Ouvrir le notebook. Seconde Anaconda Prompt : activer `formations`, se placer dans `Projet_Brulerie`, puis lancer DuckDB.

```
duckdb -ui
```

Créer un notebook et le renommer « Cours 10 ». Dans la première cellule, installer les deux extensions du cours, `iceberg` et `sqlite`, une fois pour toutes. Il faut Internet pour cette cellule seulement.

```
INSTALL iceberg;
INSTALL sqlite;
-- Attendu : la cellule passe sans rien rendre.
```

Dans la deuxième, attacher le catalogue en lecture seule, l'exécuter :

```
ATTACH IF NOT EXISTS 'plateforme/lac/iceberg/catalogue.db' AS catalogue (TYPE sqlite, READ_ONLY);
-- Attendu : catalogue paraît sous Attached databases.
```

4.2 Lire le catalogue, dans une cellule neuve.

```
SELECT table_namespace, table_name, metadata_location FROM catalogue.iceberg_tables;
-- Attendu : 1 ligne : bronze, tickets, et un chemin qui finit par
-- plateforme/lac/iceberg/bronze/tickets/metadata/00032-….metadata.json.
```

Tout le catalogue tient là : une ligne par table, et le chemin de ses métadonnées courantes. Écrire dans la table, c'est écrire des fichiers neufs, puis changer ce seul chemin : la validation se fait d'un coup. Le suffixe du nom est tiré au hasard, et sera autre chez vous.

4.3 Ranger ce chemin dans une variable. Une fonction de table n'accepte qu'une valeur fixe : `SET VARIABLE` range la valeur, et `GETVARIABLE('tickets')` la rend.

```
SET VARIABLE tickets = (SELECT metadata_location FROM catalogue.iceberg_tables WHERE table_name = 'tickets');
-- Attendu : la cellule passe sans rien rendre.
```

4.4 Compter les fichiers que la table a écrits.

```
-- 1. les fichiers de données sur le disque, ceux que la table cite, puis ses fichiers de métadonnées
SELECT (SELECT COUNT(*) FROM glob('plateforme/lac/iceberg/bronze/tickets/data/*/*.parquet')) AS donnees,
       (SELECT COUNT(*) FROM iceberg_metadata(GETVARIABLE('tickets'))) AS cites,
       (SELECT COUNT(*) FROM glob('plateforme/lac/iceberg/bronze/tickets/metadata/*.metadata.json')) AS metadonnees;
-- Attendu : 31 fichiers de données, 30 cités, 33 fichiers de métadonnées.
```


© 2026 iAlexMG. Tous droits réservés.

31 fichiers de données, un par export, rangés dans `data\jour=…\` : la partition. La table n'en cite que 30, un par jour ; celui du premier export du 12 juin est encore sur le disque, et c'est lui qui va permettre de remonter. Les métadonnées, elles, comptent une création, la partition posée, puis 31 validations, une par export : le second export du 12 juin a fait ses deux instantanés en une seule validation.

4.5 Lire l'historique autour du 12 juin.

```
SELECT sequence_number, snapshot_id, timestamp_ms, operation
FROM iceberg_snapshots(GETVARIABLE('tickets'))
WHERE sequence_number BETWEEN 11 AND 15
ORDER BY sequence_number;
-- Attendu : 5 lignes : 11 et 12 append, 13 delete, 14 et 15 append.
```

Le 12e instantané est le premier export du 12 juin ; le 13e l'efface, le 14e range le second. Les identifiants, tirés au hasard, seront autres chez vous, et les heures sont celles où la table a été écrite, aujourd'hui, en temps universel : le voyage dans le temps suit l'horloge des validations, pas celle des données.

Étape 5 — Remonter au 12 juin · RAW

5.1 Ranger les identifiants du 12e et du 13e instantané, dans une cellule neuve.

```
SET VARIABLE avant = (SELECT snapshot_id FROM iceberg_snapshots(GETVARIABLE('tickets')) WHERE sequence_number = 12);
SET VARIABLE efface = (SELECT snapshot_id FROM iceberg_snapshots(GETVARIABLE('tickets')) WHERE sequence_number = 13);
-- Attendu : la cellule passe sans rien rendre.
```

5.2 Compter les tickets du 12 juin à chaque instantané. `snapshot_from_id` fait lire la table telle qu'elle était à cet instantané.

```
-- 1. le 12 juin au 12e instantané, au 13e, puis dans la table courante
SELECT (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets'), snapshot_from_id => GETVARIABLE('avant'))
        WHERE jour = '2025-06-12') AS avant,
       (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets'), snapshot_from_id => GETVARIABLE('____'))
        WHERE jour = '2025-06-12') AS efface,
       (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets')) WHERE jour = '2025-06-12') AS maintenant;
-- Attendu : 58, 0, 65.
```

5.3 Voir de quel export venaient les 58 tickets.

```
SELECT filename, COUNT(*) AS tickets
FROM iceberg_scan(GETVARIABLE('tickets'), snapshot_from_id => GETVARIABLE('avant'))
WHERE jour = '2025-06-12'
GROUP BY filename;
-- Attendu : 1 ligne : sources\caisse\2025-06\tickets_2025-06-12_202506121542.csv, 58 tickets.
```

Le 12 juin d'avant son second export est toujours là, tel que l'export de 15 h 42 l'avait laissé. Et le 13e instantané, où le 12 juin n'a aucun ticket, aucun lecteur ne l'a vu comme table courante : le catalogue est passé du 12e au 14e d'un seul coup. On ne le lit qu'en le demandant.

© 2026 iAlexMG. Tous droits réservés.

Étape 6 — Lire la table avec DuckDB, contre le bronze · RAW

PyIceberg a écrit la table, DuckDB la lit : c'est ce qu'un format ouvert apporte, plusieurs moteurs sur les mêmes fichiers. Reste à prouver que la table courante égale le bronze du cours 02, qui ne garde que le dernier export de chaque jour.

6.1 Comparer, dans les deux sens, dans une cellule neuve.

```
-- 1. la table courante, puis ce qui manque au bronze et ce qu'il a en trop
SELECT (SELECT COUNT(*) FROM iceberg_scan(GETVARIABLE('tickets'))) AS tickets,
       (SELECT COUNT(*) FROM (SELECT * FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet')
                              EXCEPT ALL SELECT * FROM iceberg_scan(GETVARIABLE('tickets')))) AS manquent,
       (SELECT COUNT(*) FROM (SELECT * FROM iceberg_scan(GETVARIABLE('tickets'))
                              EXCEPT ALL SELECT * FROM read_parquet('plateforme/lac/bronze/tickets/*/*.parquet'))) AS en_trop;
-- Attendu : 1527 tickets, 0 et 0.
```

Les mêmes 1 527 tickets, colonne pour colonne, chemin de l'export compris. La table garde en plus son histoire : les 32 instantanés, et le fichier qui permet de les relire. C'est aussi ce qui la fait grossir. L'entretien, qui expire les vieux instantanés et efface les fichiers que plus rien ne cite, reste à Boréal.

6.2 Fermer le notebook : fermer l'onglet du navigateur, puis quitter DuckDB dans la seconde Anaconda Prompt.

```
.quit
```

Attendu : l'invite revient, et la seconde fenêtre peut se fermer.

Étape 7 — Versionner · RAW

La table et son catalogue vivent hors du dépôt, dans `plateforme\lac\iceberg\` : ce sont des données, pas du code.

7.1 Valider le script, dans la première fenêtre.

```
git -C 01_de add iceberg.py
git -C 01_de commit -m "Cours 10 : les tickets de la caisse en table Iceberg, un instantané par export"
REM Attendu : 1 file changed.
git -C 01_de status
REM Attendu : nothing to commit, working tree clean.
```

À retenir

- Une table ouverte, ce sont des fichiers Parquet et des métadonnées qui disent lesquels comptent ; le catalogue tient le chemin des métadonnées courantes, et une écriture se valide en changeant ce seul chemin.
- Chaque écriture fait un instantané, et chaque instantané se relit : le 12 juin d'avant son second export est toujours là, 58 tickets.

© 2026 iAlexMG. Tous droits réservés.

- Un format ouvert se lit par d’autres moteurs que celui qui l’écrit : DuckDB lit la table de PyIceberg, et elle égale le bronze du cours 02.

© 2026 iAlexMG. Tous droits réservés.